Java Programming: Classes and Exceptions

Desenvolvimento de Software e Sistemas Móveis (DSSMV)
Licenciatura em Engenharia de Telecomunicações e Informática
LETI/ISEP

2026/27

Paulo Baltarejo Sousa and Carlos Filipe Freitas
{pbs,caf}@isep.ipp.pt



Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Outline

1. **Introducing classes**
   - Class Fundamentals
2. **Naming Conventions & Comments**
3. **A Closer Look at Methods**
4. **Inheritance & Polymorphism**
5. **Handling Exceptions**
6. **Threads**
7. **Example: School Application**

Introducing classes

What is a class?

- The **class** is at the core of Java.
  - **It is the logical construct upon which the entire Java language is built** because **it defines the shape and nature of an object**.
  - The class **forms the basis for object-oriented programming in Java**.
- Any concept you wish to implement in a Java program must be encapsulated within a class.
- A **class should represent a single concept from a problem domain**, such as business science, or mathematics.
  - Some of the classes are used to represent concepts from mathematics:
    - Point
    - Rectangle
    - Ellipse
  - Other classes are abstractions of real-life entities:
    - BankAccount
    - CashRegister
    - Vehicle

Classes and objects

- Perhaps the most important thing to understand about **a class** is that **it defines a new data type**.
- Once defined, **this new type can be used to create objects of that type**.
  - Thus, a **class is a template for an object**, and an **object** is an **instance of a class**.
    - Because an object is an instance of a class, you will often see the two words object and instance used interchangeably.

The General Form of a Class (I)

- A class is declared by use of the `class` keyword.
  - The name for a class should be a noun that describes the concept.

```
class classname {
//******************* Variables ******************
   type instance-variable1;
   type instance-variable2;
// ...
   type instance-variableN;
//****************** Constructors ******************
   classname(parameter-list) {
// body of constructor
   }
//******************* Methods ******************
   type methodname1(parameter-list) {
// body of method
   }
   type methodname2(parameter-list) {
// body of method
   }
// ...
   type methodnameN(parameter-list) {
// body of method
   }
}
```

The General Form of a Class (II)

- The data, or variables, defined within a class are called **instance variables** or **attributes**.
- The **code is contained within methods**.
- The **methods** and **variables** defined within a **class** are called **members of the class**.
- The **instance variables** are acted upon and accessed by the **methods** defined for that class.
  - **It is the methods that determine how a class' data can be used**.
- A **constructor** initializes an object immediately upon creation.

`main` method

Java classes do not need to have a `main` method. You only specify one if that class is the starting point for your program.

A Simple Class (I)

- Here is a class called `Box` that defines three instance variables: `width`, `height`, and `depth`.

```
class Box {
   double width;
   double height;
   double depth;
}
```

- To create a `Box` object, you will use a statement like the following:

```
Box mybox = new Box(); // create a Box object called mybox
```

- Every `Box` object will contain its own copies of the instance variables `width`, `height`, and `depth`.
- To access instance variables, you will use the dot (`.`) operator, which links the name of the object with the name of an instance variable.

```
mybox.width = 100;
```

A Simple Class (II)

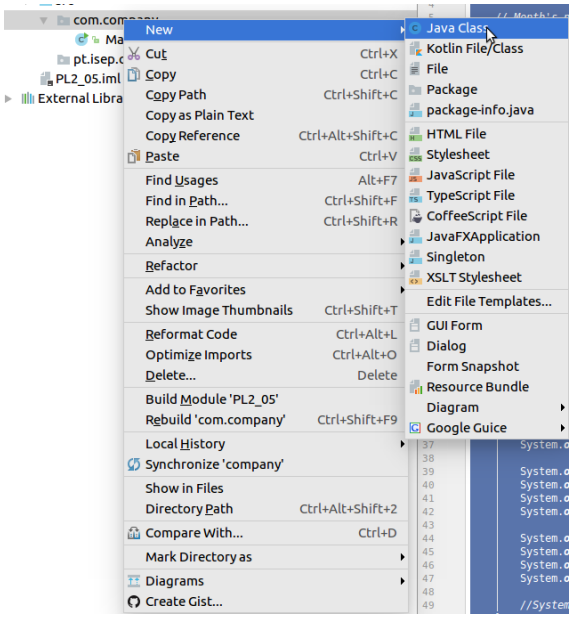
- **Tester program** that creates two `Box` objects.

```
class Main {
   public static void main(String[] args) {
      Box mybox1 = new Box();
      Box mybox2 = new Box();
      double vol;
// assign values to mybox1's instance variables
      mybox1.width = 10;
      mybox1.height = 20;
      mybox1.depth = 15;
/* assign different values to mybox2's
instance variables */
      mybox2.width = 3;
      mybox2.height = 6;
      mybox2.depth = 9;
// compute volume of first box
      vol = mybox1.width * mybox1.height * mybox1.depth;
      System.out.println("Volume is " + vol);
// compute volume of second box
      vol = mybox2.width * mybox2.height * mybox2.depth;
      System.out.println("Volume is " + vol);
   }
}
```

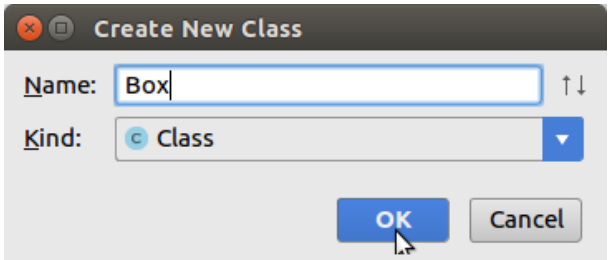
Check: `TP02_01`

IntelliJ: create a class

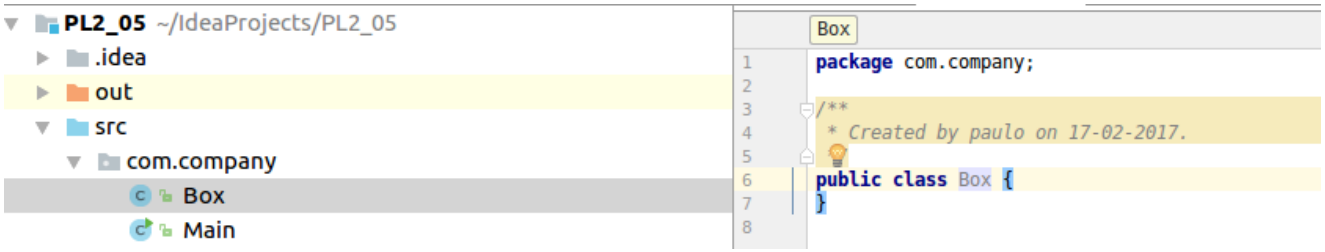
1. Right click on package name



2. Enter class name



3. Edit class

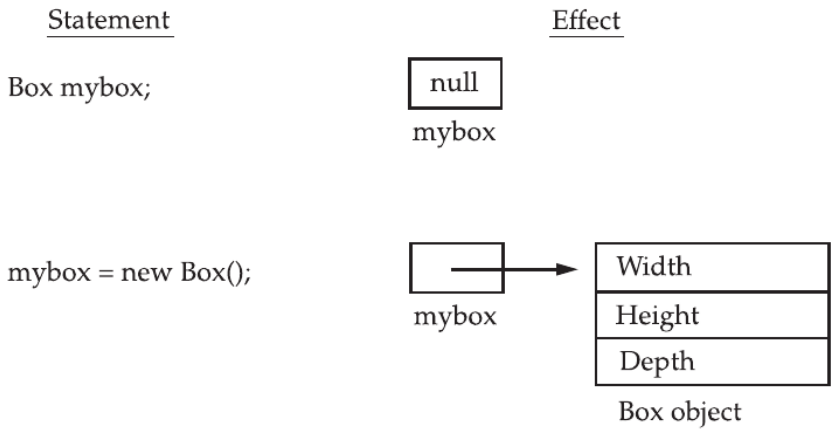


Creating Objects

- **Creating objects** of a class is a two-step process:
  - First, you must **declare a variable** of the class type.
    - This variable does not define/create an object.
    - Instead, it is simply a variable that can refer to an object.
  - Second, **you must acquire a physical copy of the object and assign it to that variable**, using the `new` operator.
    - The `new` operator dynamically **allocates** (that is, allocates at run time) **memory for an object and returns a reference to it**.
    - This reference is, more or less, the address in memory of the object allocated by `new` and it is then stored in the variable.

```
// declare reference to object
Box mybox;
// allocate a Box object
mybox = new Box();
```

```
// declare reference to object and
     allocate a Box object
Box mybox = new Box();
```



Constructor

- A **constructor initializes an object immediately upon creation**.
- It has **the same name as the class** in which it resides.
- It is automatically **called immediately after the object is created, before the `new` operator completes**.
- **Constructors have no return type**, not even void.
  - This is because the implicit return type of a class' constructor is the class type itself.

```
class Box {
   double width;
   double height;
   double depth;
   // This is the constructor for Box.
   Box() {
      System.out.println("Constructing Box");
      width = 10;
      height = 10;
      depth = 10;
   }
}
```

Check: `TP02_02`

Parameterized Constructors

- While the `Box` constructor in the preceding slide does initialize a `Box` object.
  - It is not very useful–all boxes have the same dimensions.
- What is needed is a way to construct `Box` objects of various dimensions.
- The **easy solution is to add parameters to the constructor**.

```
class Box {
   double width;
   double height;
   double depth;
   Box() {
      width = 10;
      height = 10;
      depth = 10;
   }
   Box(double w, double h, double d) {
      width = w;
      height = h;
      depth = d;
   }
}
```

Tester program

- A tester program that creates two `Box` objects.

```
class Main {
   public static void main(String[] args) {
      Box mybox1 = new Box();
      Box mybox2 = new Box(3, 6, 9);
      double vol;
// compute volume of first box
      vol = mybox1.width * mybox1.height * mybox1.depth;
      System.out.println("Volume is " + vol);
// compute volume of second box
      vol = mybox2.width * mybox2.height * mybox2.depth;
      System.out.println("Volume is " + vol);
   }
}
```

Check: `TP02_03`

Introducing Methods (I)

- A Java **method is a collection of statements that are grouped together to perform an operation**.
- This is the general form of a method:

```
type name(parameter-list) {
// body of method
}
```

  - `type` specifies the type of data returned by the method.
    - It can be any valid type, including class types.
    - If the method does not return a value, its return type must be `void`.
  - The name of the method is specified by `name`.
  - The `parameter-list` is a sequence of type and identifier pairs separated by commas.
    - Parameters are essentially variables that receive the value of the arguments passed to the method when it is called.
    - If the method has no parameters, then the parameter list will be empty.

Introducing Methods (II)

- Methods that have a return type other than `void` return a value to the calling routine using the following form of the return statement:

```
return value;
```

  - `value` is the value returned.
- Adding a method to `Box` class

```
class Box {
   double width;
   double height;
   double depth;
   double getVolume() {
      return (width * height * depth);
   }
}
```

Introducing Methods (III)

- A tester program that creates two `Box` objects.
- Invoking `getVolume` method.

```
class Main {
   public static void main(String[] args) {
      Box mybox1 = new Box();
      Box mybox2 = new Box(3, 6, 9);
      double vol;
// get volume of first box
      vol = mybox1.getVolume();
      System.out.println("Volume is " + vol);
// get volume of second box
      vol = mybox2.getVolume();
      System.out.println("Volume is " + vol);
   }
}
```

Check: `TP02_04`

The `this` Keyword

- Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the `this` keyword.
- `this` can be used inside any method to **refer to the current object**.

```
// Use this to resolve name-space collisions.
Box(double width, double height, double depth) {
   this.width = width;
   this.height = height;
   this.depth = depth;
}
```

The `static` keyword (I)

- To create static a member, precede its declaration with the keyword `static`.
- When a member is declared `static`, **it can be accessed before any objects of its class are created**, that is, **without reference to any object**.
- You can declare both methods and variables to be `static`.
  - The most common example of a static member is `main`.
    - `main` is declared as `static` because it must be called before any objects exist.
  - Instance variables declared as `static` are, essentially, global variables.
    - When objects of its class are declared, **no copy of a `static` variable is made**.
    - Instead, **all instances of the class share the same static variable**.

The `static` keyword (II)

- Static methods, which have the `static` modifier in their declarations, **should be invoked with the class name**, without the need for creating an instance of the class.

```
public class Utils {
   public static int getNumber(String label){
      ...
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      int n = Utils.getNumber("Enter an id");
   }
}
```

The `static` keyword (III)

- **A `static` variable belongs to the class, not to any object of the class**.
- Example: implementing a counter.
  - We want to assign bank account numbers sequentially, the first one, with number 1001, the next with number 1002, and so on.
  - To solve this problem, we need to have a single value of `lastAssignedNumber` that is a property of the class, not any object of the class.

```
public class BankAccount{
   private int accountNumber;
   private static int lastAssignedNumber = 1000;

   public BankAccount(){
      lastAssignedNumber++;
      accountNumber = lastAssignedNumber;
   }
   ...
}
```

The `final` keyword

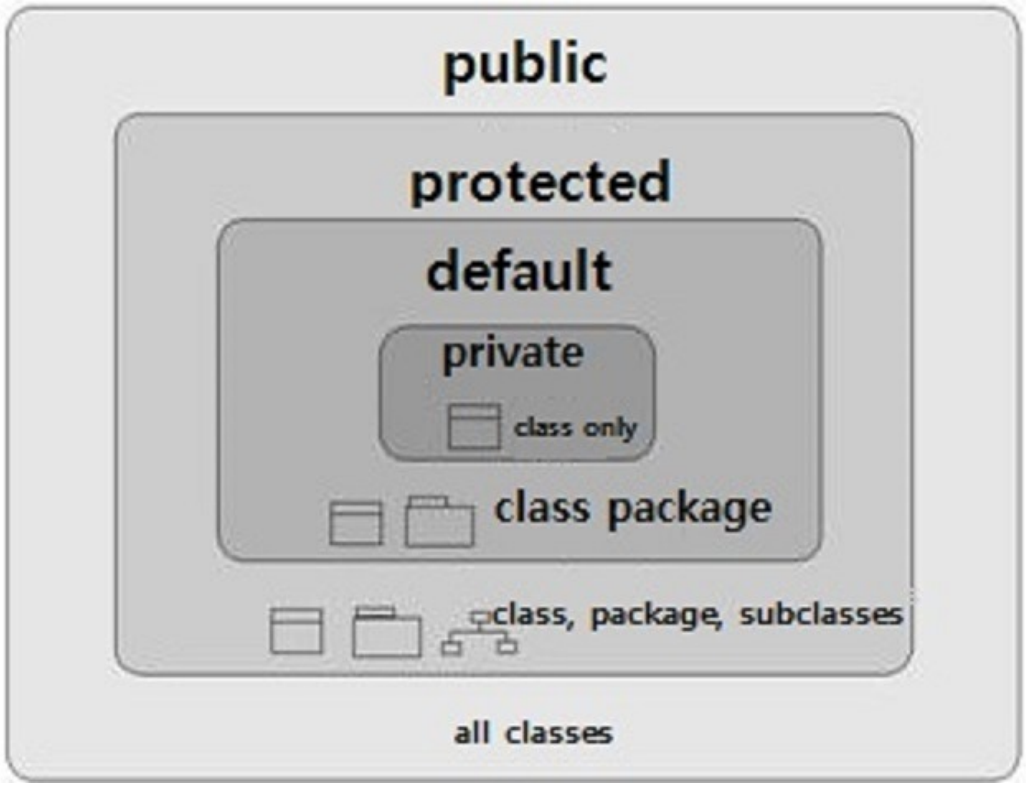
- A variable can be declared as `final`, which means that you must initialize a final variable when it is declared, and doing so prevents its contents from being modified.
- A `final` variable **is a constant**.
- Frequently, constant values are needed in several methods.
  - Then you **should declare them together with the instance variables of a class and tag them as `static` and `final`**.
    - `final` indicates that the value is a constant.
    - `static` means that the constant belongs to the class

```
public class Math
{
   . . .
   public static final double E = 2.7182818284590452354;
   public static final double PI = 3.14159265358979323846;
}
```

```
double circumference = Math.PI * diameter;
```

Access modifier (I)

- Access modifier is a general term which **may refer to a class, member variable, method or constructor**:
- Access modifiers **are applied to features to specify which other classes can access them**.



Access modifier (II)

- `public`
  - The least restrictive access modifier is `public`.
    - A feature (i.e. class, variable or method) that has been marked as `public` will be accessible by any other class.
- `private`
  - The most restrictive access modifier is `private`.
    - A feature that has been marked as `private` will only be accessible by an instance of the class it is has been defined in.
- `protected`
  - The `protected` access modifier is more restrictive than `public` but less restrictive than `private`.
    - A feature that has been marked as `protected` will only be accessible by subclasses of the class or a class in the same package.
- `Default`
  - A feature that has not been explicitly specified with any access modifier, then it will be accessible from classes in the same package as itself.

Naming Conventions & Comments

CamelCase

- Java follows **CamelCase** syntax for naming.
- CamelCase is a naming convention in which a name is formed of multiple words that are joined together as a single word with the **first letter of each of the multiple words capitalized** so that each word that makes up the name can easily be read.
  - The name derives from the hump or humps that seem to appear in any CamelCase name.
- In **UpperCamelCase**, **the first letter of the new word is upper case**, allowing it to be easily distinguished from a **lowerCamelCase** name, in which **the first letter of the first name is lower case**.
- The advantage of CamelCase is that in any computer system where the letters in a name have to be contiguous (no spaces), a more meaningful name can be created using a descriptive sequence of words without violating the naming limitation.

Classes and Methods

- Classes
  - UpperCamelCase
  - Class names should be nouns
    - Try to keep your class names simple and descriptive.
    - Use whole words-avoid acronyms and abbreviations.

```
class Box;
class BowWeight;
```

- Methods
  - lowerCamelCase.
  - Method **names should be verbs**.

```
run();
runFast();
getBackground();
```

Variables and Constants

- Variables
  - lowerCamelCase.
  - Variable names should not start with underscore _ or dollar sign $ characters, even though both are allowed.

```
int i;
char c;
float myWidth;
```

- Constants
  - The names of variables declared as constants should be **all uppercase with words separated by underscores ("_")**

```
static final int MIN_WIDTH = 4;
static final int MAX_WIDTH = 999;
static final int GET_THE_CPU = 1;
```

Interfaces and Packages

- Interfaces
  - UpperCamelCase, like class names.

```
interface List;
interface Storing;
```

- Packages
  - It should be in lowercase letters.
  - The prefix of a unique package name is always written in all-lowercase ASCII letters and should be one of the top-level domain names (pt, com, and etc).
  - Subsequent components of the package name vary according to an organization's own internal naming conventions.

```
com.company
pt.ipp.isep.cmovdis
```

Three types of comments

- **Line comments** start with two forward slashes (`//`) and continue to the end of the current line.
- **Block comments** start with a forward slash and an asterisk (`/*`) and end with an asterisk and a forward slash (`*/`).
- **Javadoc comments** are a special case of block comments called documentation comments because they are used to automatically generate nicely formatted program documentation with a program named `javadoc`.
  - Javadoc comments begin with a forward slash and two asterisks (`/**`) and end with an asterisk and a forward slash (`*/`).

A Closer Look at Methods

Overloading Methods (I)

- **Overloading**
  - When **two or more methods within the same class that share the same name**, as long as their **parameter declarations are different**.
  - Overloaded methods may have different **return types**
    - The return type alone **is insufficient to distinguish two versions of a method**.
- When an **overloaded method is invoked**:
  - It is used the **type** and/or **number of arguments** as its guide to determine which version of the overloaded method to actually call.
    - The **return type is not considered**.

Overloading Methods (II)

```
class OverloadDemo {
   void test() {
      System.out.println("No parameters");
   }
// Overload test for one integer parameter.
   void test(int a) {
      System.out.println("a: " + a);
   }
// Overload test for two integer parameters.
   void test(int a, int b) {
      System.out.println("a and b: " + a + " " + b);
   }
// overload test for a double parameter
   double test(double a) {
      System.out.println("double a: " + a);
      return a*a;
   }
}
```

Overloading Constructors

```
class Box {
   double width;
   double height;
   double depth;
// constructor used when all dimensions specified
   Box(double w, double h, double d) {
      this.width = w;
      this.height = h;
      this.depth = d;
   }
// constructor used when no dimensions specified
// use -1 to indicate an uninitialized box
   Box() {
      this.width = -1;
      this.height = -1;
      this.depth = -1;
   }
// constructor used when cube is created
   Box(double len) {
      this.width = this.height = this.depth = len;
   }
}
```

Using Objects as Parameters

- Methods and constructors

```
class Box {
   double width;
   double height;
   double depth;
   // This is constructor for Box that receive an object.
   Box(Box obj) {
      width = obj.width;
      height = obj.height;
      depth = obj.depth;
   }
   // return true if o is equal to the invoking object
   boolean equals(Box obj) {
      if( this. width == obj.width && this.height == obj.height && this.depth == obj.
            depth)
         return true;
      else
         return false;
   }
}
```

Check: `TP02_05`

A Closer Look at Argument Passing

- Passing by **value**.
  - This approach copies the value of an argument into the formal parameter of the method.
  - Therefore, **changes made to the parameter of the method have no effect on the argument**.
- Passing by **reference**.
  - In this approach, a reference to an argument is passed to the parameter.
  - This means that **changes made to the parameter will affect the argument used to call the method**.
- In Java
  - When you pass a **primitive type** to a method, **it is passed by value**.
  - When you pass an **object** to a method, **it is passed by reference**.

Check: `TP02_06`

A Closer Look at Returning Types (I)

- **A method can return any type of data**, including class and primitive types as well as `void`.

```
public class Box {
   private double width;
   private double height;
   private double depth;
   public Box(double width, double height, double depth) {
      this.width = width;
      this.height = height;
      this.depth = depth;
   }
   public double getWidth() {
      return width;
   }
   public void setWidth(double width) {
      this.width = width;
   }
   public double getVolume() {
      return (width * height * depth);
   }
   public Box getBoxObject() {
      Box box = new Box(10,10, 10);
      return box;
   }
}
```

A Closer Look at Returning Types (II)

- Returning a **value**

```
public class Box {
...
   public double getWidth() {
      return width;
   }
   ...
}
```

- Returns an object **reference**
  - All objects are dynamically allocated in the heap using `new` operator.
    - The object will continue to exist as long as there is a reference to it somewhere in your program.

```
public class Box {
...
   public Box getBoxObject() {
      Box box = new Box(10,10, 10);
      return box;
   }
}
```

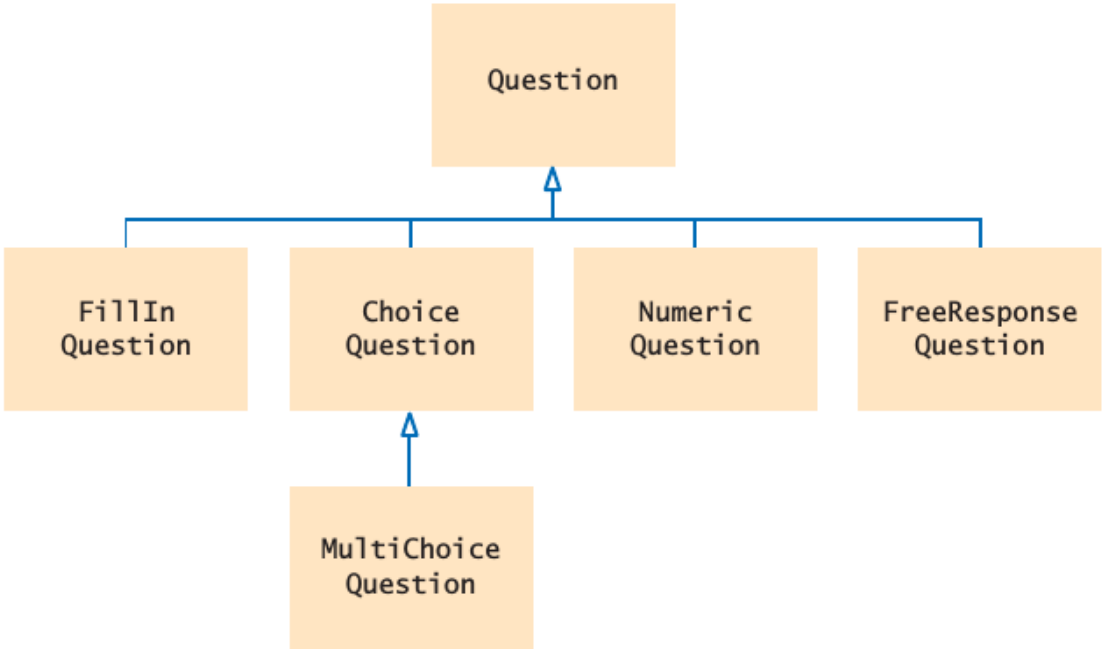
Inheritance & Polymorphism

Defining a class

```
package packagename
modifier class classname {
//******************* Variables ******************
modifier type instance-variable1;
modifier type instance-variable2;
// ...
modifier type instance-variableN;
//******************* Constructors ******************
modifier classname(parameter-list) {
// body of constructor
}
//******************* Methods ******************
modifier type methodname1(parameter-list) {
// body of method
}
modifier type methodname2(parameter-list) {
// body of method
}
// ...
modifier type methodnameN(parameter-list) {
// body of method
}
}
```

Deriving a class

- In object-oriented design, **inheritance** is a relationship between a more general class (called the **superclass**) and a more specialized class (called the **subclass**).
- The subclass inherits data and behavior from the superclass.



Implementing Subclasses

- The `extends` reserved word indicates that a class inherits from a superclass.
- A subclass can **override** a superclass method by providing a new implementation.
- A subclass **inherits** all methods that it does not override.

Syntax

```
public class SubclassName extends SuperclassName
{
   instance variables
   methods
}
```

The reserved word extends denotes inheritance.

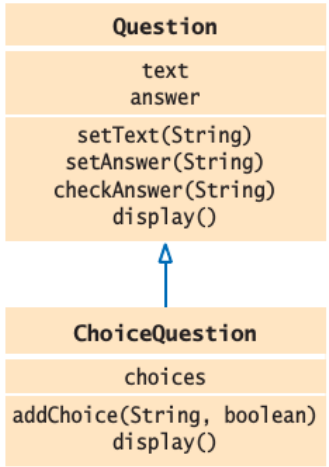
Subclass / Superclass

```
public class ChoiceQuestion extends Question
{
   private ArrayList<String> choices;
   public void addChoice(String choice, boolean correct) { . . . }
   public void display() { . . . }
}
```

Declare instance variables that are added to the subclass.

Declare methods that are added to the subclass.

Declare methods that the subclass overrides.



- Which methods are inherited?
- Which methods are overriden?

Calling the Superclass Constructors and Methods

- To call a superclass constructor or method, use the `super` reserved word.

Syntax

```
public ClassName(parameterType parameterName, . . .)
{
   super(arguments);
   . . .
}
```

The superclass constructor is called first.

The constructor body can contain additional statements.

```
public ChoiceQuestion(String questionText)
{
   super(questionText);
   choices = new ArrayList<String>;
}
```

If you omit the superclass constructor call, the superclass constructor with no arguments is invoked.

Syntax

```
super.methodName(parameters);
```

Calls the method of the superclass instead of the method of the current class.

```
public void deposit(double amount)
{
   transactionCount++;
   super.deposit(amount);
}
```

If you omit super, this method calls itself.

See page 437.

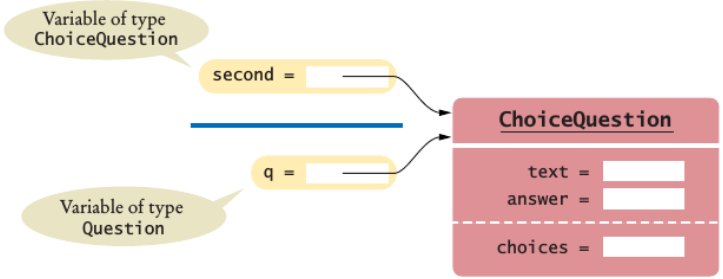
Dynamic Method Lookup

- A **subclass reference** can be used when a **superclass reference** is expected.

```
public static void presentQuestion(Question q){
q.display();
...
}
```

```
ChoiceQuestion second = new ChoiceQuestion();
. . .
presentQuestion(second); // OK to pass a ChoiceQuestion
```

- When the `presentQuestion` method executes, the object references stored in `second` and `q` refer to the same object of type `ChoiceQuestion`.



Upcasting & Downcasting

- **Upcasting is casting to a supertype**, while **downcasting is casting to a subtype**.
- Upcasting **is always allowed**.
  - It is correct to **store a subclass reference** in a **superclass variable**

```
ChoiceQuestion cq = new ChoiceQuestion();
Question q = cq; // OK
Object anObject = cq; // OK
```

- Downcasting involves a type check (requires the `instanceof` operator) and can throw a `ClassCastException`.
  - **store a superclass reference** in a **subclass variable**

```
Object anObject = new Question();

if(anObject instanceof Question){
Question q = (Question) anObject;
}
```

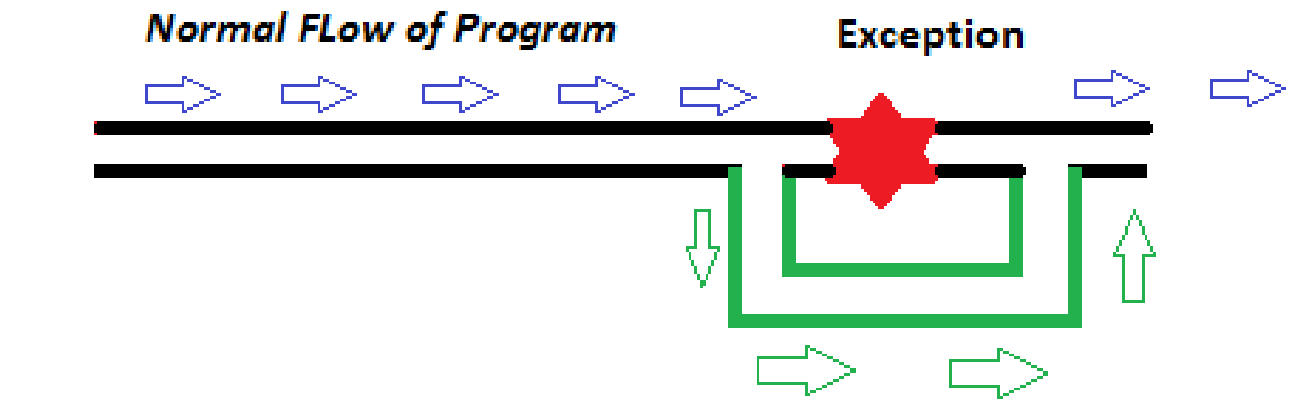
Handling Exceptions

Exception

- An **exception** is an unexpected or error condition.
  - A program might issue a command to read a file from a disk, but the file does not exist there.
  - A program might attempt to write data to a disk, but the disk is full or unformatted.
  - A program might ask for user input, but the user enters an invalid data type.
  - A program might attempt to divide a value by 0.
  - A program might try to access an array with a subscript that is too large or too small.

Exception Handling

- **It provides a flexible mechanism for passing control from the point of error detection to a handler that can deal with the error.**



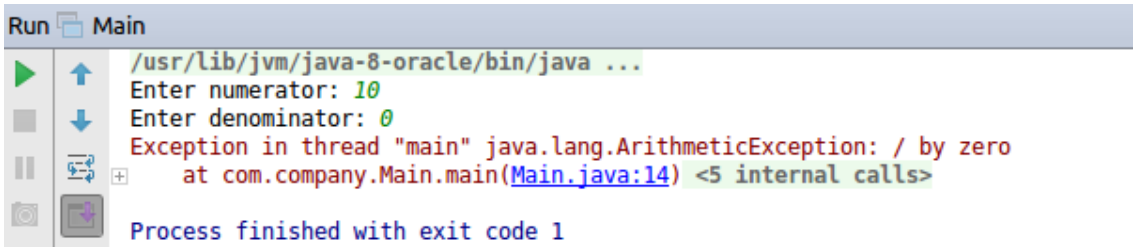
- When **an exception is throwable**, **execution does not continue with the next statement**, but with an **exception handler**.

Uncaught (I)

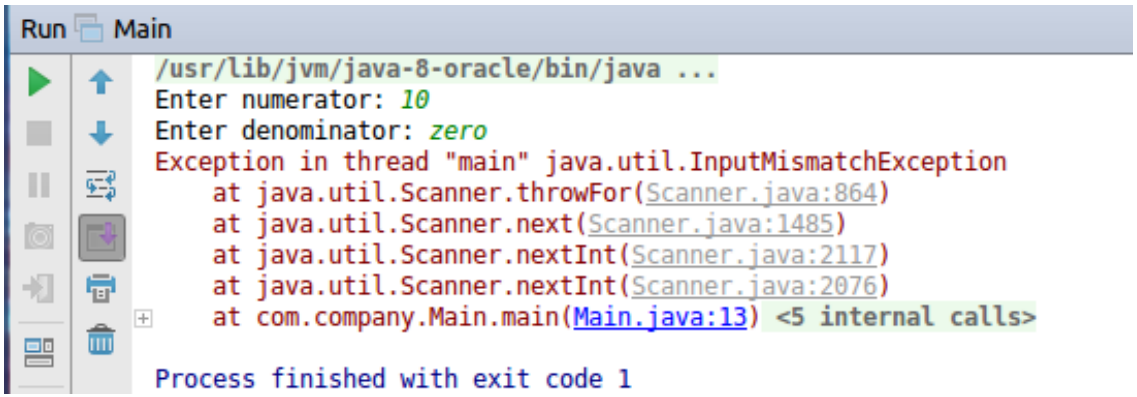
```
public class Main {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
int numerator, denominator, result;
do {
System.out.print("Enter numerator: ");
numerator = input.nextInt();
System.out.print("Enter denominator: ");
denominator = input.nextInt();
result = numerator / denominator;
System.out.println(numerator + " / " + denominator + " = " + result);
}while(numerator != 0);
}
}
```

Uncaught (II)

- Division by 0;



- Bad input;



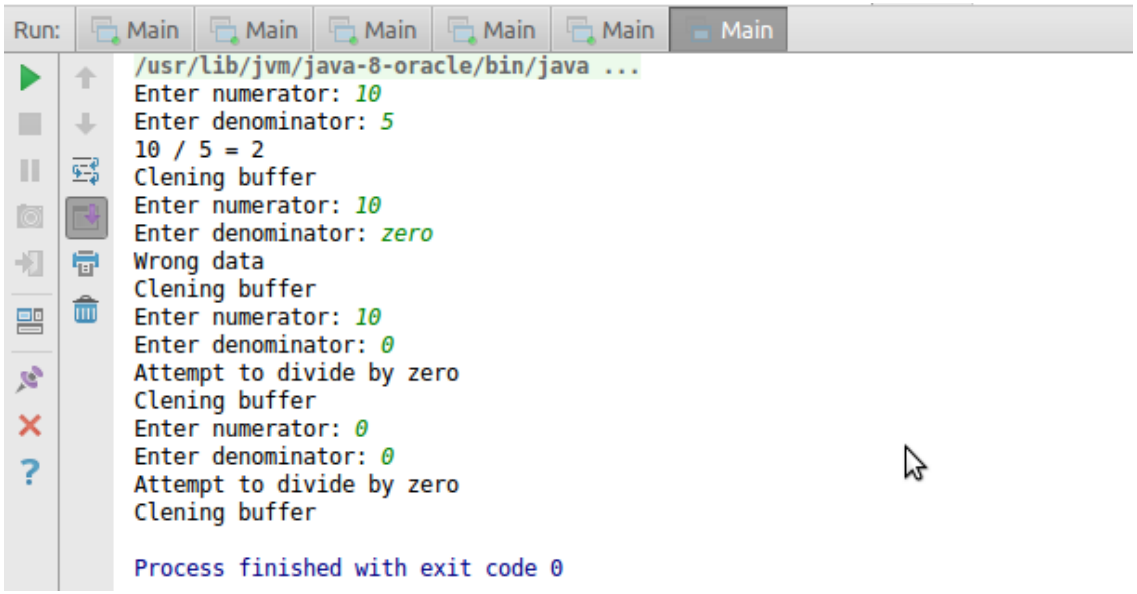
Check: `TP3_05.zip`

Caught (I)

```
public class Main {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
int numerator, denominator, result;
do {
try {
System.out.print("Enter numerator: ");
numerator = input.nextInt();
System.out.print("Enter denominator: ");
denominator = input.nextInt();
result = numerator / denominator;
System.out.println(numerator + " / " + denominator + " = " + result);
} catch (ArithmeticException e) { //specific for arithmetic
System.out.println("Attempt to divide by zero");
} catch (InputMismatchException e) { //specific bad input
System.out.println("Wrong data ");
}catch (Exception e) { //this is generic. It could be used to manage any exception
System.out.println("Other exception: " + e.toString());
}finally{
System.out.println("Clening buffer");
input.nextLine();
}
}while(numerator != 0);
}
}
```

Caught (II)

- Division by 0;
- Bad input;



- Besides clarity, an advantage to object-oriented exception handling is the flexibility it allows in the handling of error situations.

Check: `TP3_06.zip`

Exception Specification

- If a method throws an exception that it will not catch, but such exception will be caught by a different method.
  - You must use the keyword `throws` followed by an `Exception` type in the method header.

```
public class Main {
  private static final double[] price = {15.99, 27.88, 34.56, 45.89};
  public static void main(String[] args) {
  Scanner input = new Scanner(System.in);
  System.out.print("Enter item number: ");
  int item = input.nextInt();
  try{
  displayPrice(item);
  }
  catch(IndexOutOfBoundsException e){
  System.out.println("Price is $0");
  }
  }
  static void displayPrice(int item) throws IndexOutOfBoundsException {
  System.out.println("The price is $" + price[item]);
  }
}
```

Check: `TP02_07`

Custom Exception (I)

```
public class InvalidPriceException extends IllegalArgumentException{
  public InvalidPriceException(String s) {
  super(s);
  }
  }
```

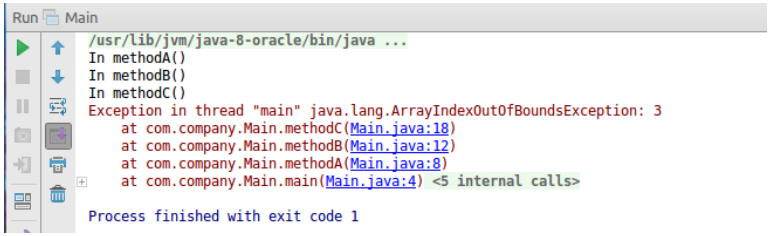
```
  public class Product {
  private int id;
  private String description;
  private int price;
  ...
  public void setPrice(int price) {
  if(price > 0){
  this.price = price;
  }else{
  throw new InvalidPriceException(price+ " is not a valid price");
  }
  }
}
```

Custom Exception (II)

```
public static Product getProduct(){
Scanner kbd = new Scanner(System.in);
boolean flag = false;
Product product = new Product();
System.out.print("Id: ");
int id = kbd.nextInt();
kbd.nextLine();
product.setId(id);
System.out.print("Description: ");
String description = kbd.nextLine();
product.setDescription(description);
int price = -1;
do {
try {
flag = false;
System.out.print("Price: ");
int price = kbd.nextInt();
kbd.nextLine();
product.setPrice(price);
}catch (InvalidPriceException e){
flag = true;
}
}while(flag && price < 0);
return product;
}
```

Stack trace

```
1  package com.company;
2  public class Main {
3  public static void main(String[] args) {
4  methodA();
5  }
6  static void methodA(){
7  System.out.println("In methodA()");
8  methodB();
9  }
10 static void methodB() {
11 System.out.println("In methodB()");
12 methodC();
13 }
14 static void methodC() {
15 System.out.println("In methodC()");
16 int [] array = {0, 1, 2};
17 //This cause an out of bound exception
18 System.out.println(array[3]);
19 }
20 }
```



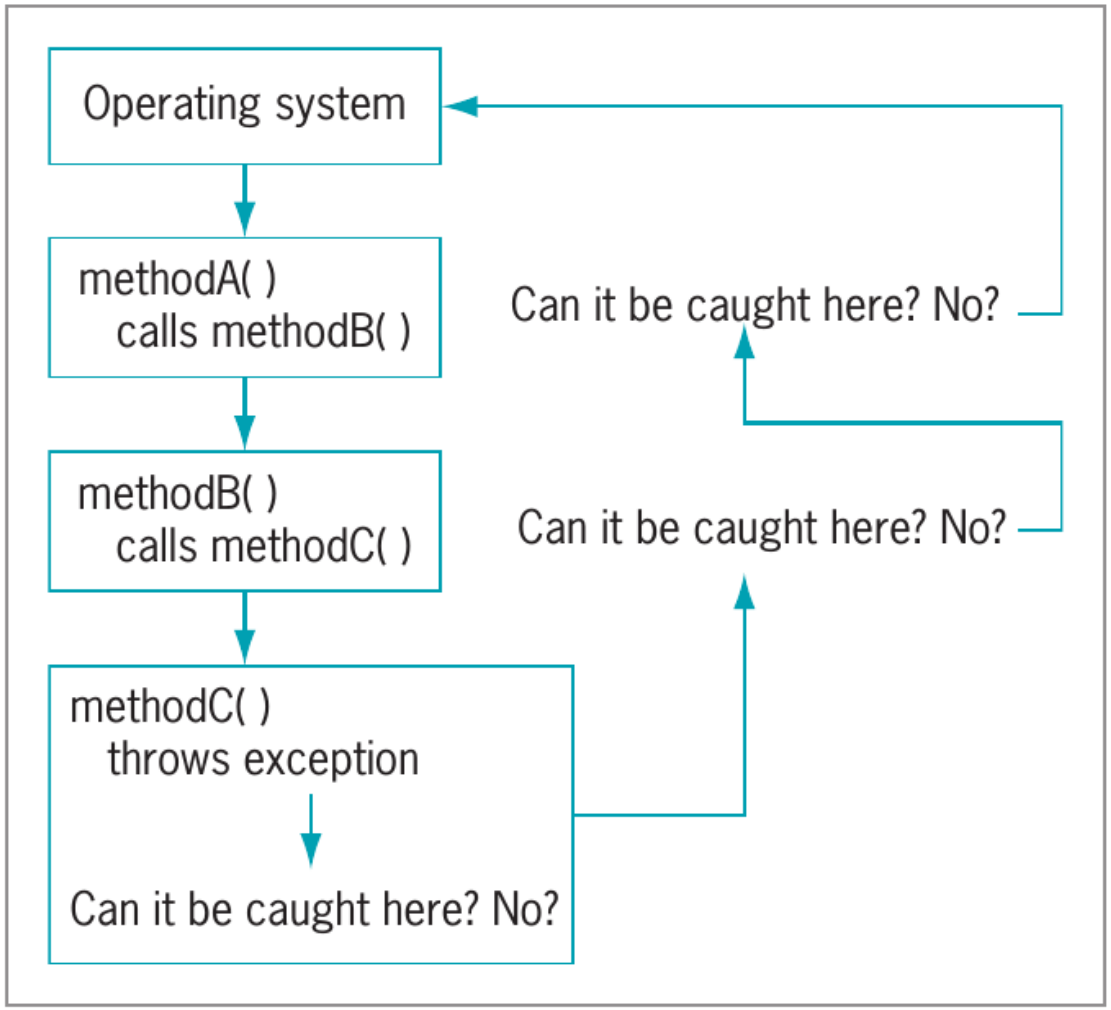
- The list of error messages after each attempted execution is called a **stack trace**.
  - The list shows each method that was called as the program ran.

Check: `TP02_08`

Tracing Exceptions (I)

- How the call stack works:
  - If `methodA()` calls `methodB()`, and `methodB()` calls `methodC()`, and `methodC()` throws an exception, Java first looks for a catch block in `methodC()`.
  - If none exists, Java looks for the same thing in `methodB()`.
  - If `methodB()` does not have a catch block, Java looks to `methodA()`.
  - If `methodA()` cannot catch the exception, it is thrown to the Java Virtual Machine, which displays a message at the command prompt.

Tracing Exceptions (II)



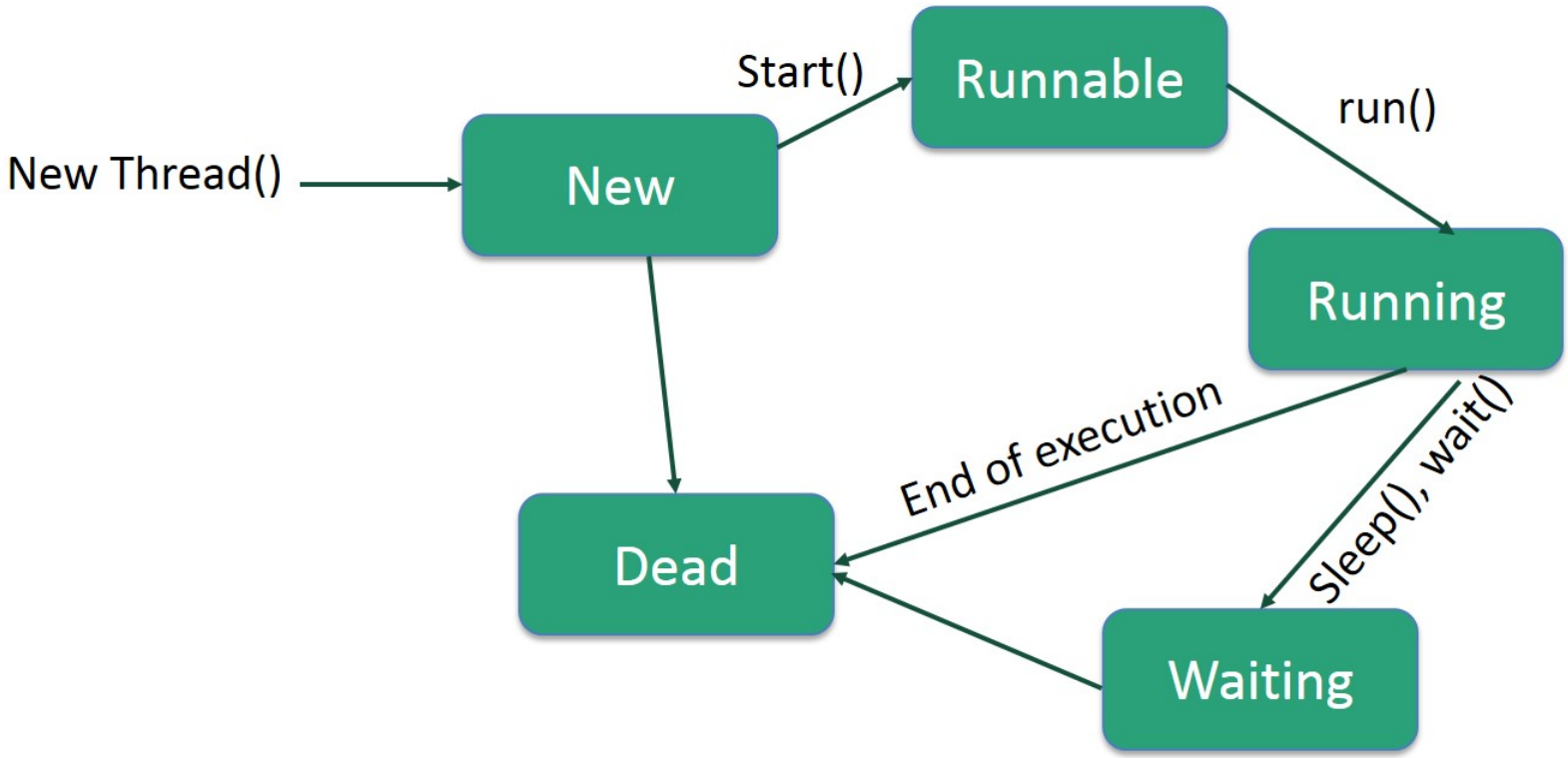
Threads

Multitasking & Multithread

- Java is a **multi-threaded** programming language which means we can develop multi-threaded program using Java.
- A multi-threaded program **contains two or more parts that can run concurrently** and each part can handle a different task at the same time making optimal use of the available resources specially when your computer has multiple CPUs.
- **Multitasking** is when multiple processes share common processing resources such as a CPU.
- Multi-threading extends the idea of multitasking into applications where you can subdivide specific operations within a single application into individual threads.
- Each of the threads can run in parallel.
  - The OS divides CPU processing time not only among different threads of each application.
- Multi-threading enables you to write in a way where multiple activities can proceed concurrently in the same program.

Life Cycle of a Thread

- A thread goes through various stages in its life cycle.
  - For example, a thread is born, started, runs, and then dies.



Check: `TP02_09`

Check: `TP02_10`

Example: School Application

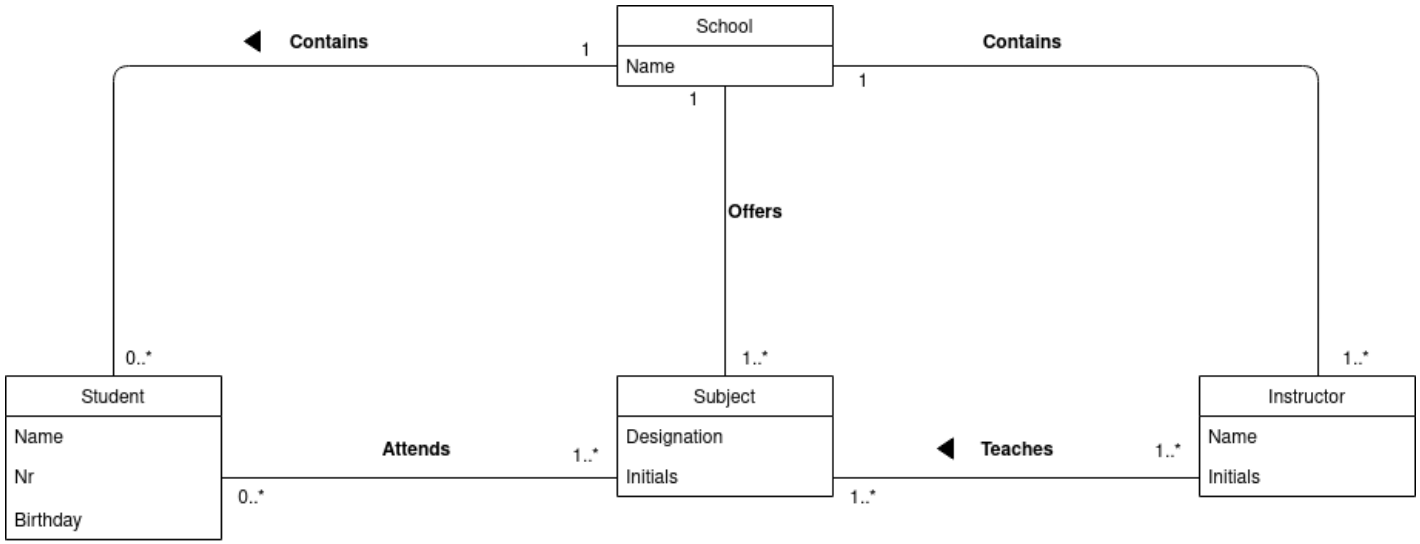
Context/Problem

- Software Application for a school.
- Features
  - School has students, instructors and offers subjects.
  - Instructors can teach many subjects.
  - The same subject can be taught by different instructors.
  - Student can attend to many subjects.
- Requirements
  - Data must be persistent.
  - User Interface must be console-based.
- User Stories
  - As a system user, I want to print (in paper) the student subject marks, so that I can issue the certificate.
  - As a school manager, I want to award the best student, so that I can motivate them.
  - As a school manager, I want to award the youngest student.

Check: `SchoolApplication.zip`

Analysis: Domain Model (I)

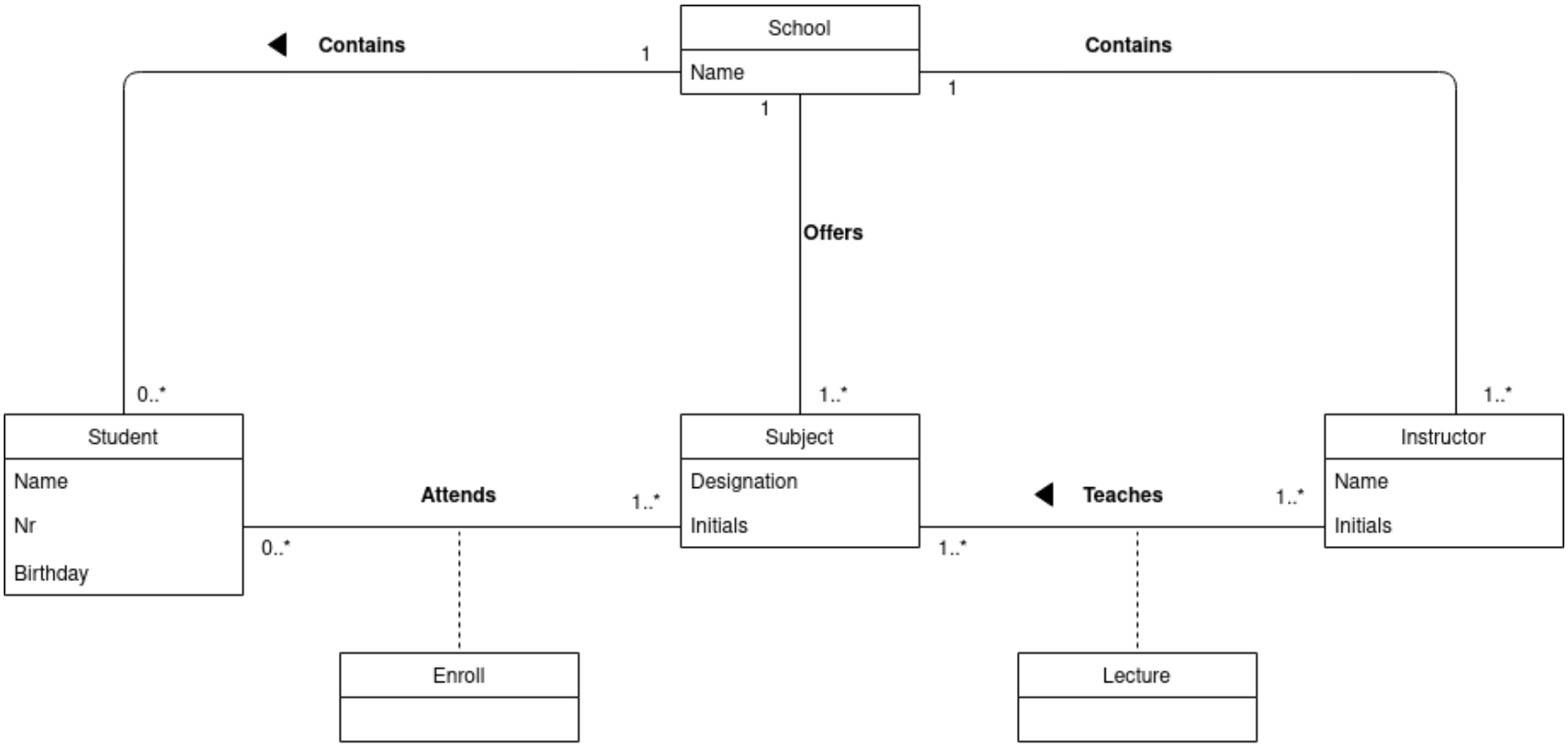
- **Entities** and **Relations**
  - `School` contains `Student`, `Instructor` entities and offers `Subject`.
  - `Instructor` entities can teach many `Subject` entities.
  - The same `Subject` entity can be taught by different `Instructor` entities.
  - `Student` entity can attend to many `Subject` entities.



- However, this model has **many-to-many** relations

Analysis: Domain Model (II)

- Breaking **many-to-many relation** with an associative class.
  - `Enroll` for breaking `Student` and `Subject`
  - `Lecture` for breaking `Instructor` and `Subject` relation



Analysis: Non-Functional Requirements (I)

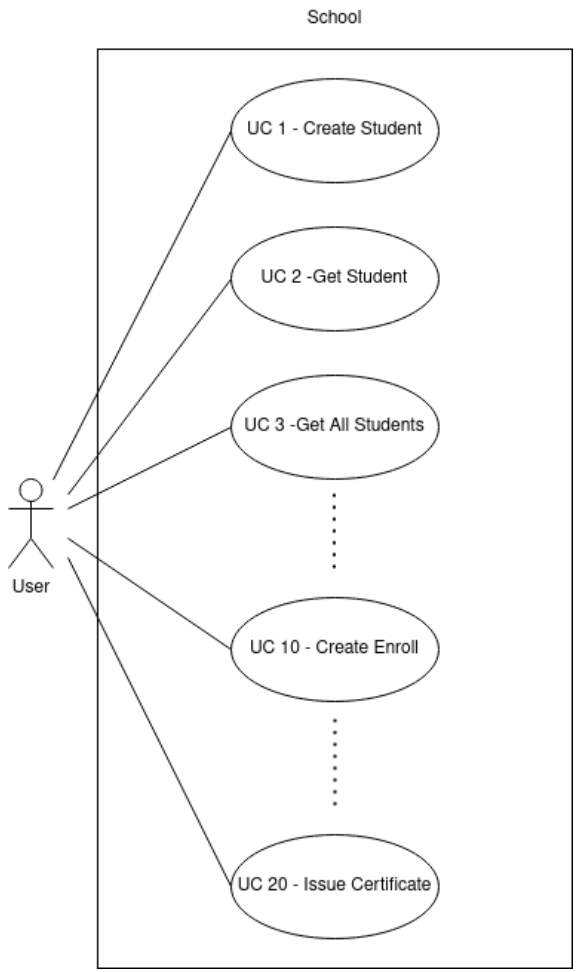
- **Usability**
  - Console-based user interface
- **Reliability**
  - No
- **Performance**
  - No
- **Supportability**
  - Huge amount of tests (unit tests)

Analysis: Non-Functional Requirements (II)

- +
  - **Design constraints**
    - Data persistent in binary files
  - **Implementation constraints**
    - Java language
  - **Interface constraints**
    - No

Analysis: Functional Requirements (I)

- Funcionalities



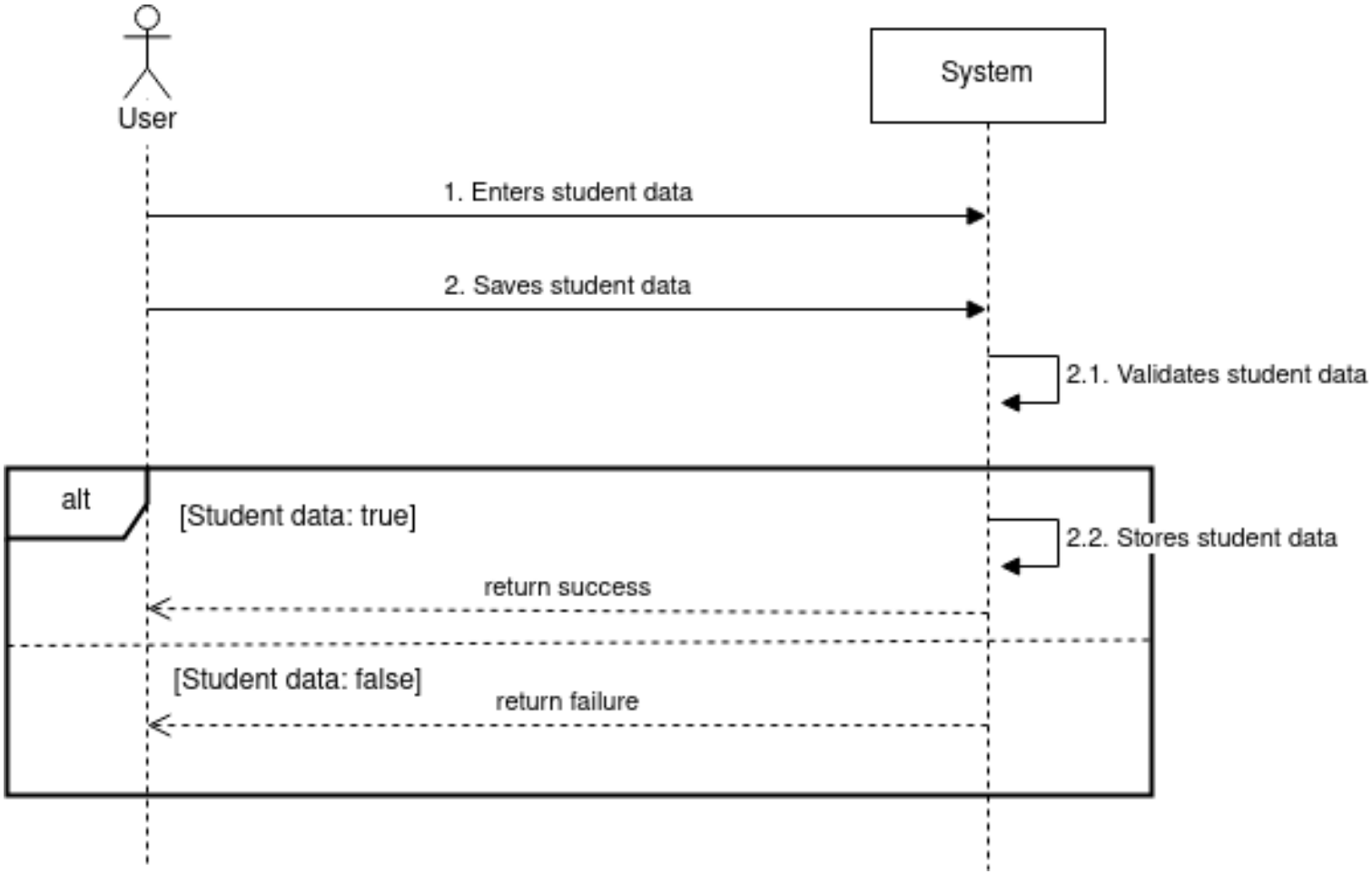
Analysis: Functional Requirements (II)

- Specification: UC 1 - Create Student

Description	Add one student to the system
Pre-condition	There are no preconditions associated with this use case.
Post-condition	The number of students has to increase one.
Basic path	1. The User enters the student data. 2. The User saves the student data. 3. The System validates the student data. 4. The System stores the student data. 5. The System returns success.
Alternative path	• Invalid data • The System returns failure.

Analysis: Functional Requirements (III)

- SSD: UC 1 - Create Student



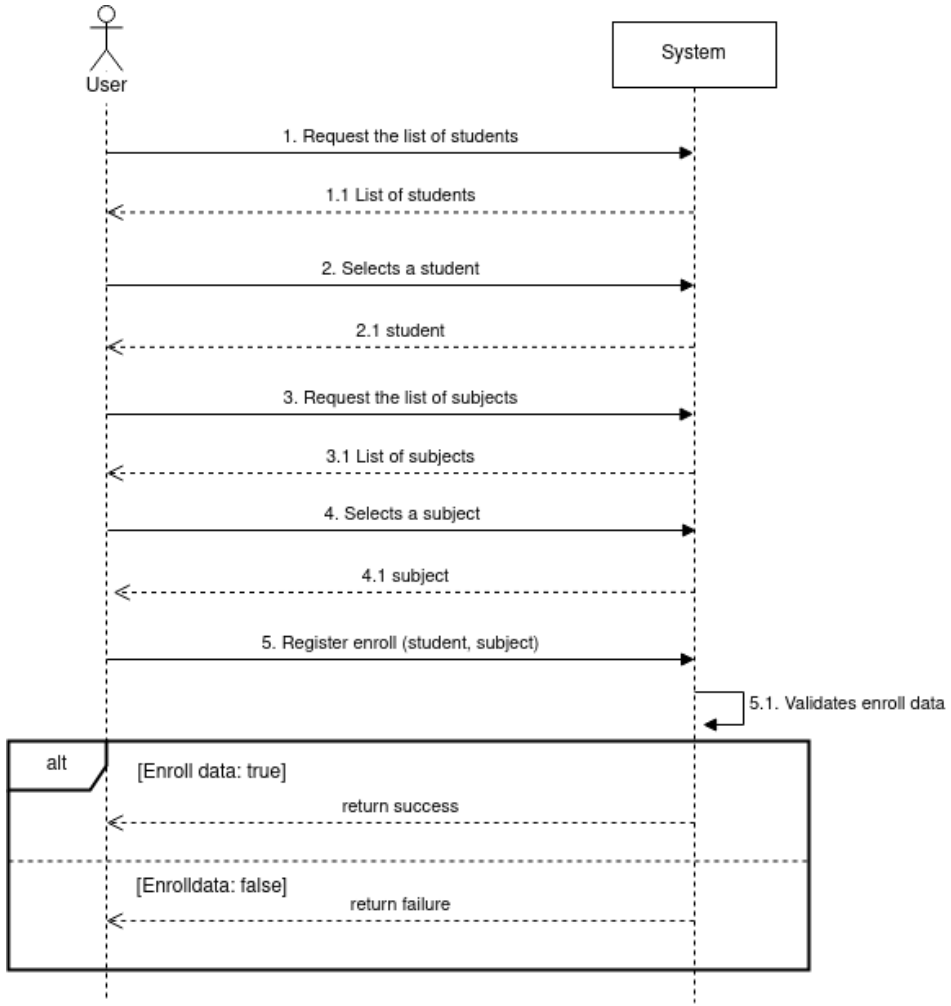
Analysis: Functional Requirements (IV)

- Specification: UC 10 - Create Enroll

Description	Associate one student to one subject
Pre-condition	The student and the subject must exist in the system
Post-condition	One more Enroll.
Basic path	1. The User requests the list of students. 2. The User selects one student. 3. The User requests the list of subjects. 4. The User selects one subject. 5. The User register the enroll. 6. The System validates the enroll data. 7. The System stores the enroll. 8. The System returns success.
Alternative path	• Invalid data • The System returns failure.

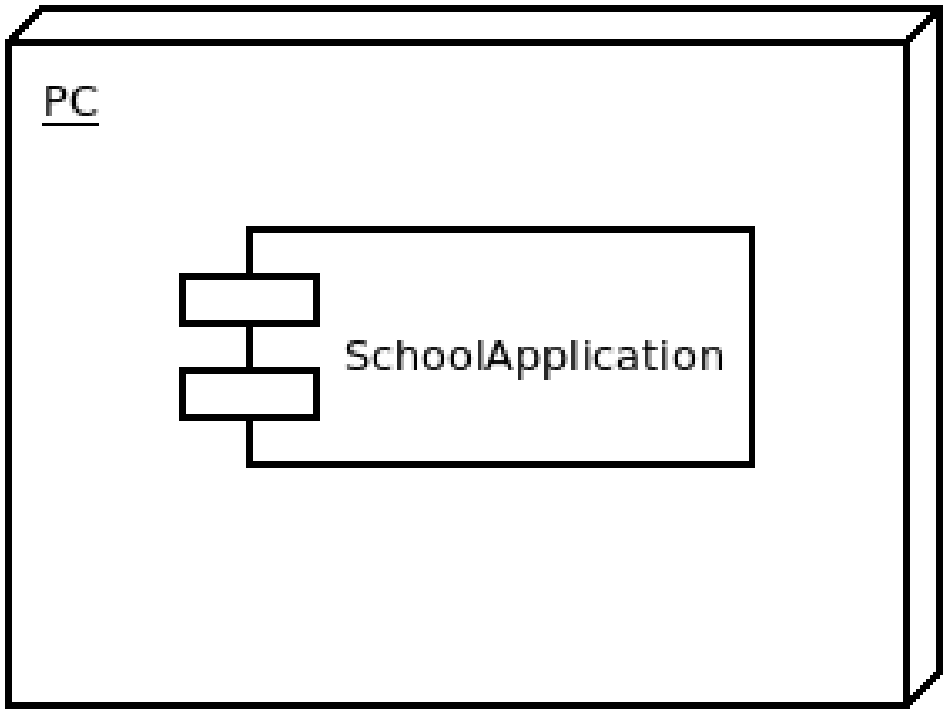
Analysis: Functional Requirements (V)

- SSD: UC 10 - Create Enroll



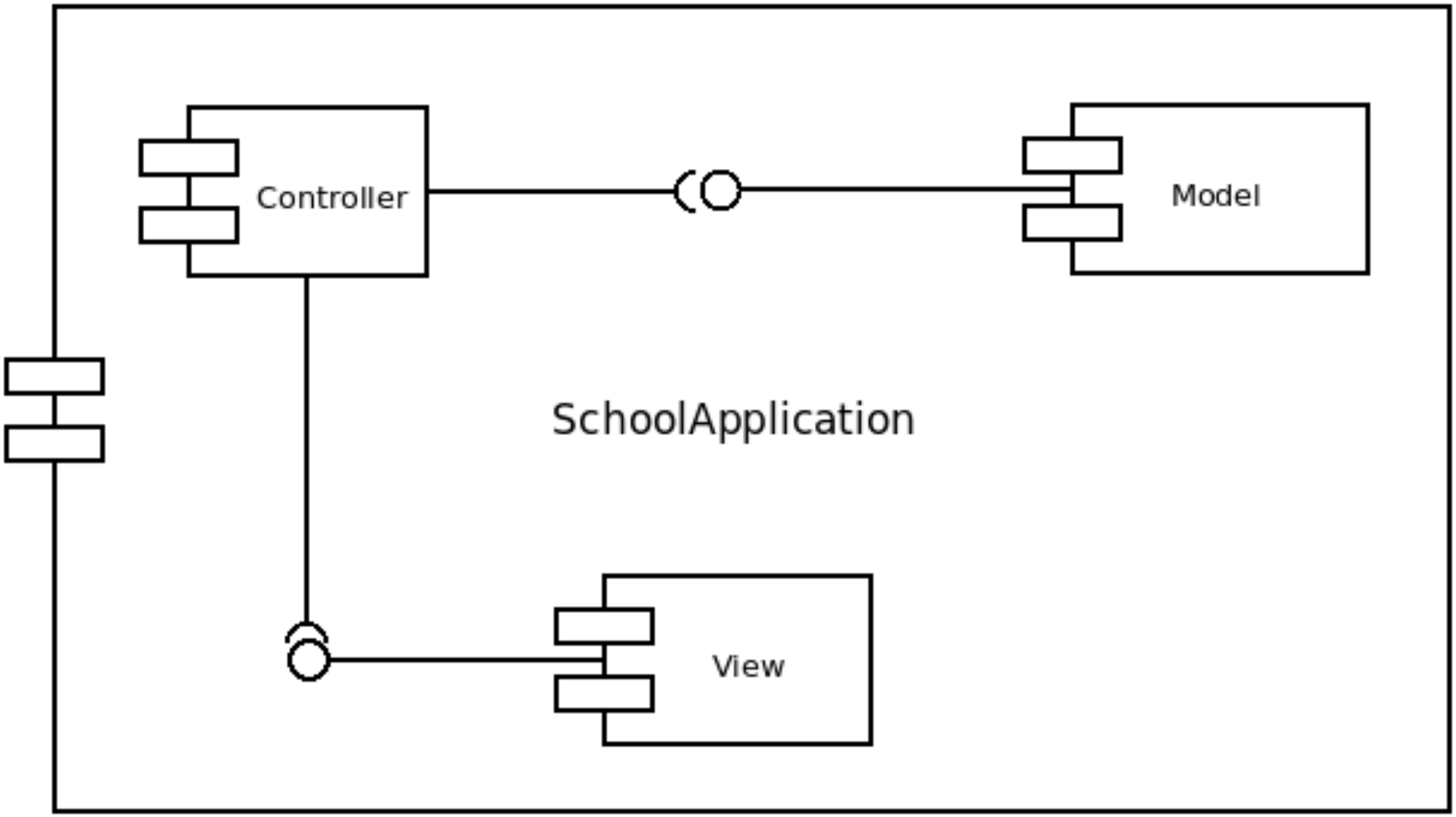
Design: Physical Architecture

- Usage of deployment diagram



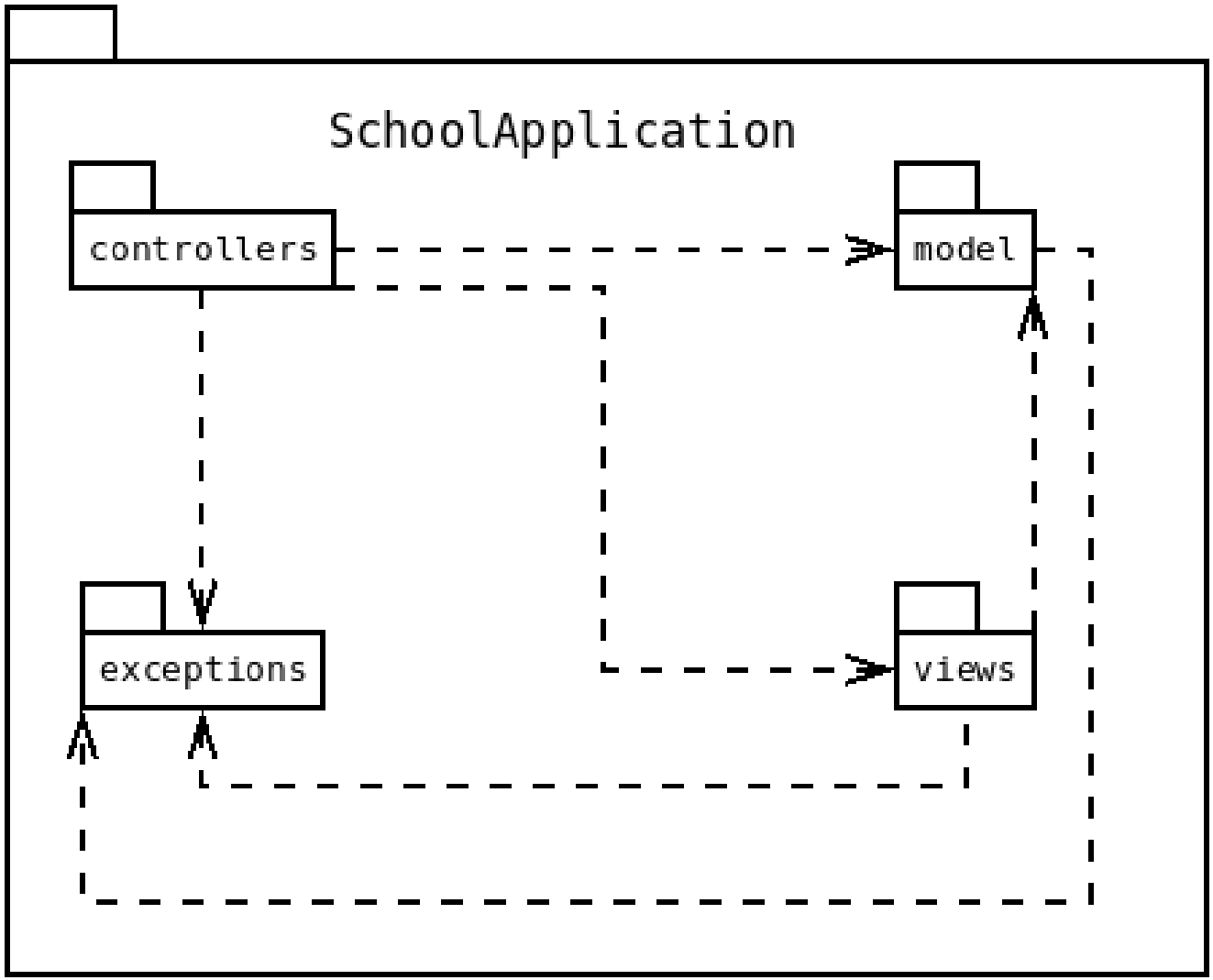
Design: Logical Architecture

- Usage of component diagram



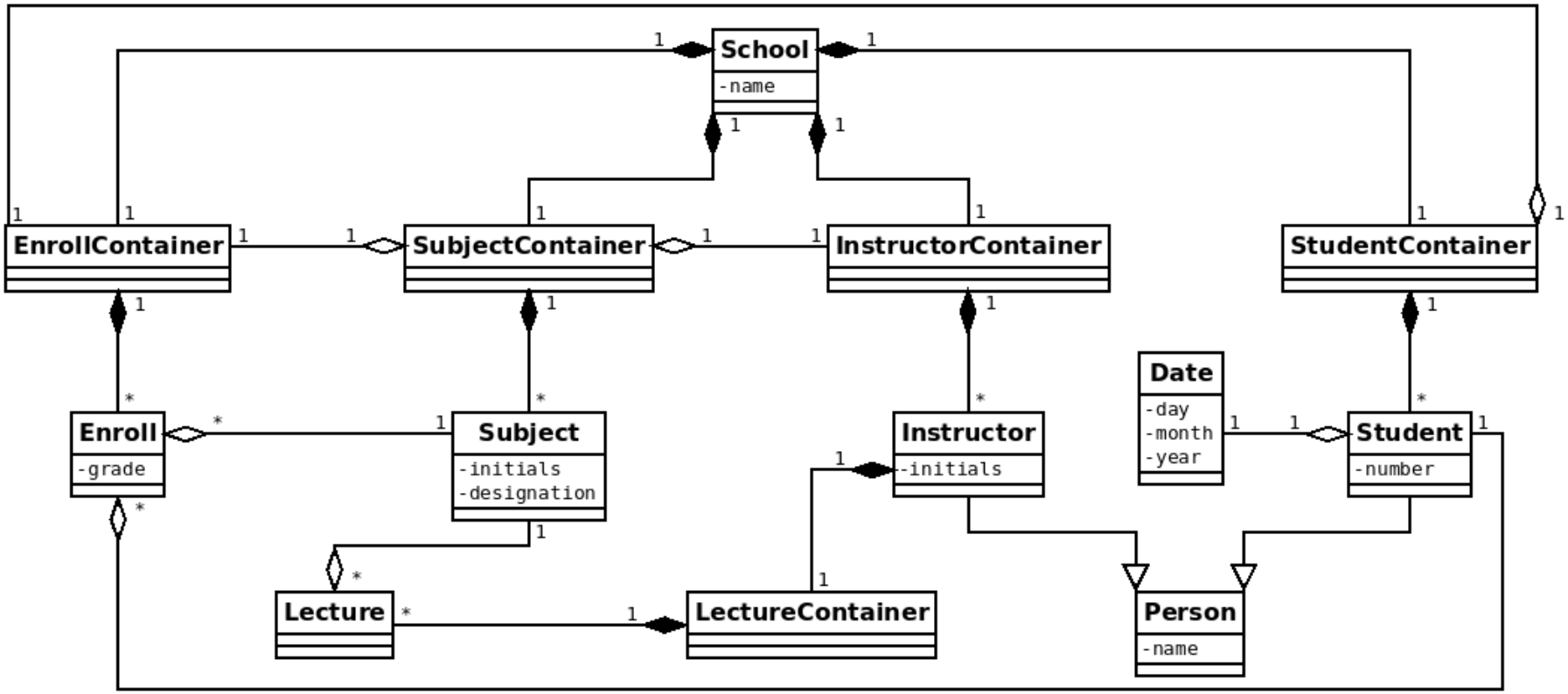
Design: Code Organization

- Usage of package diagram



Design: `model` Class Diagram

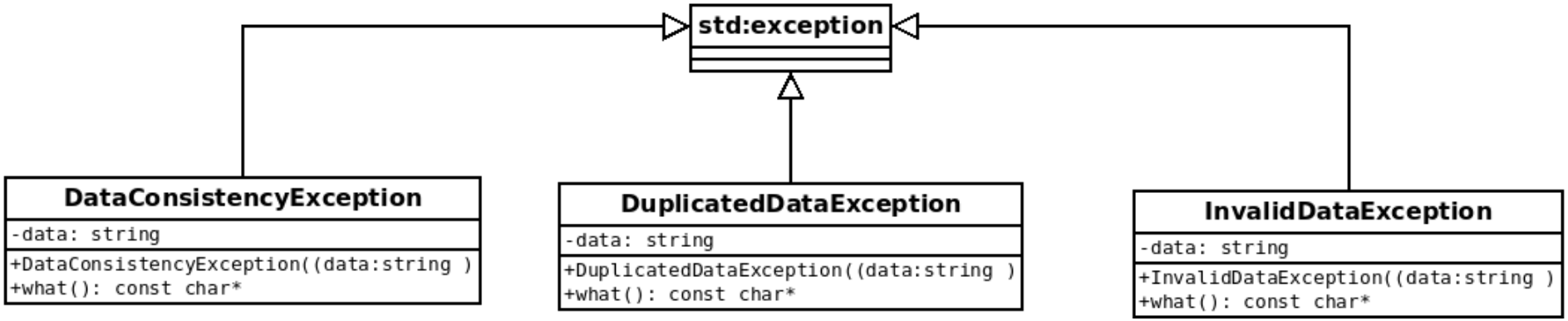
- Usage of class diagram



- It is missing:
  - Functions and attribute types.
  - Relations with exceptions' package classes.

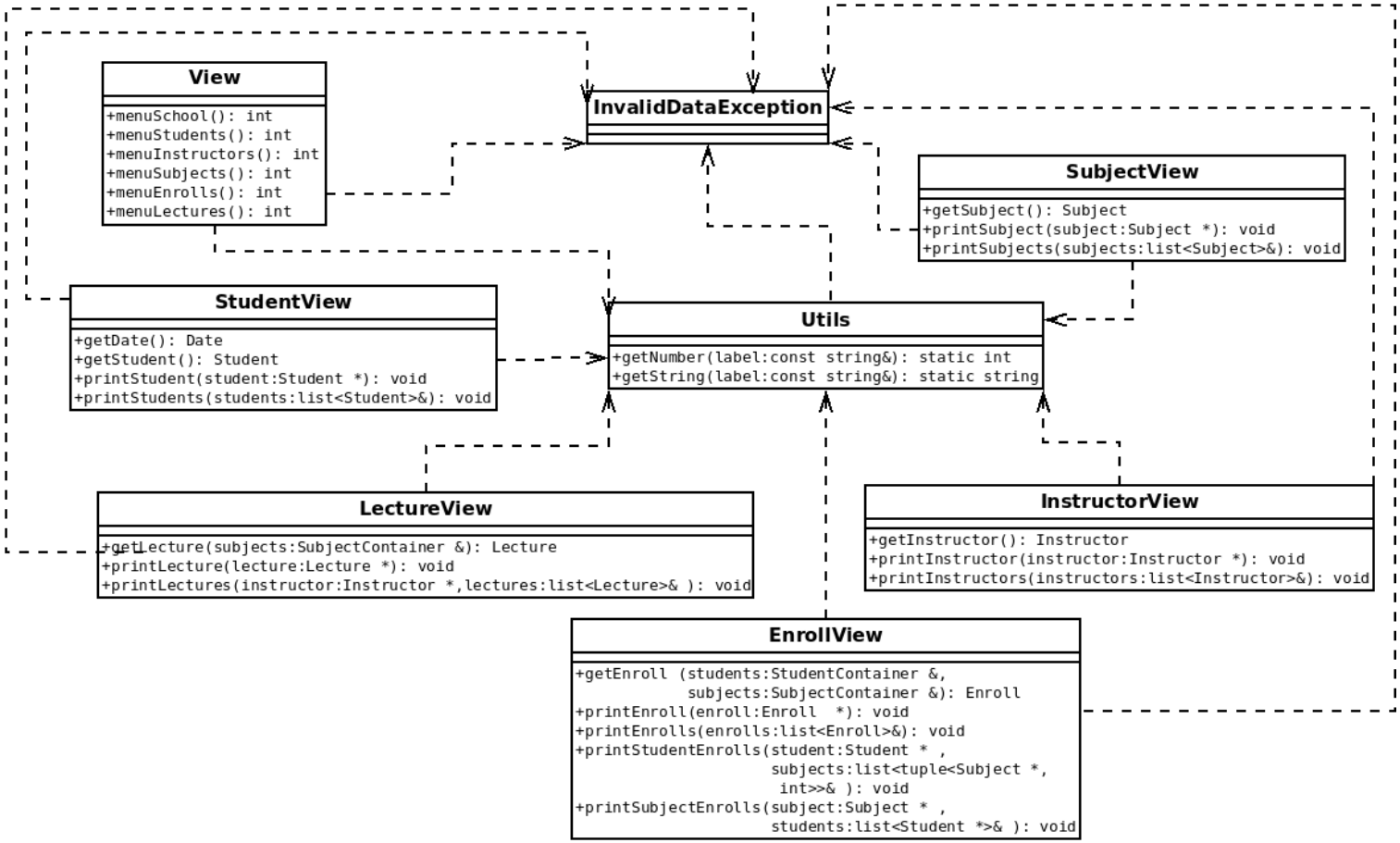
Design: `exceptions` Class Diagram

- Usage of class diagram



Design: `views` Class Diagram

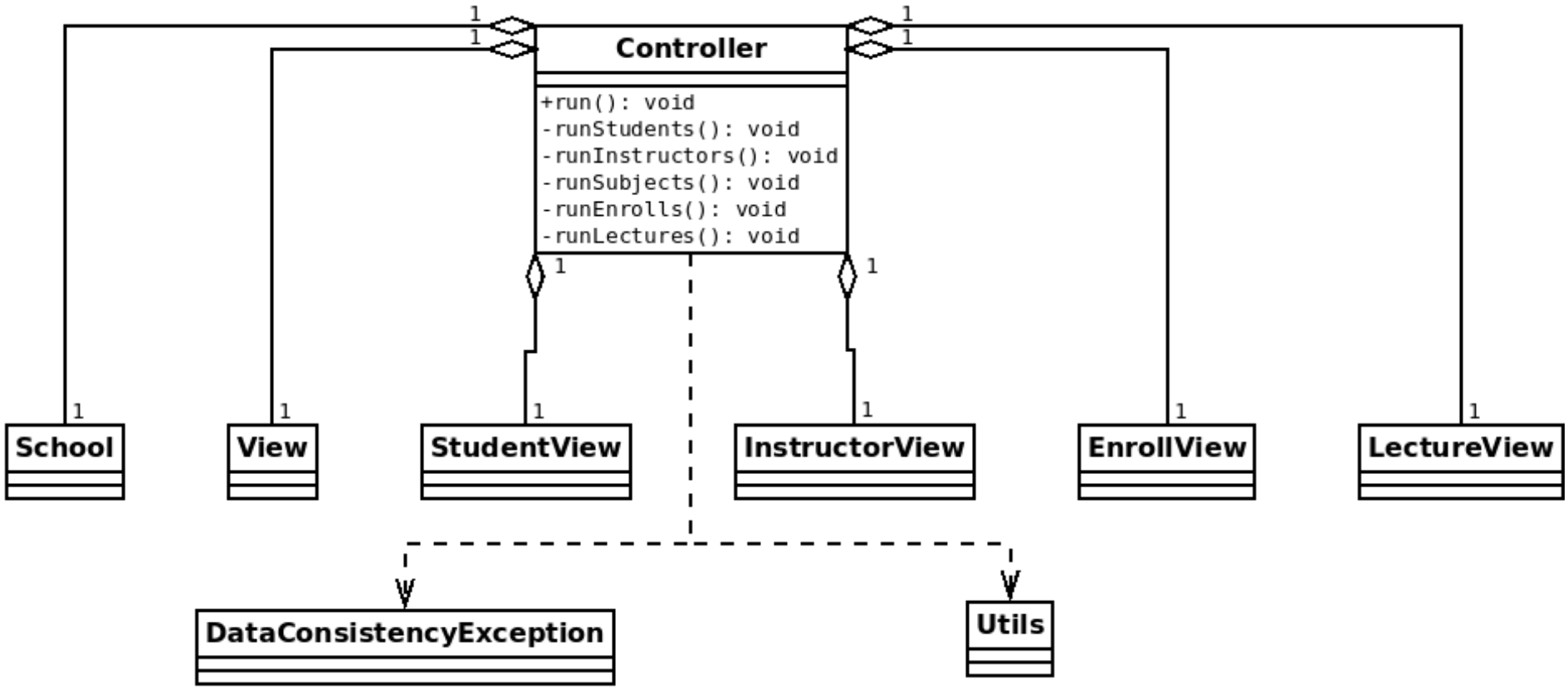
- Usage of class diagram



- It is missing:
  - Relations with model's package classes.

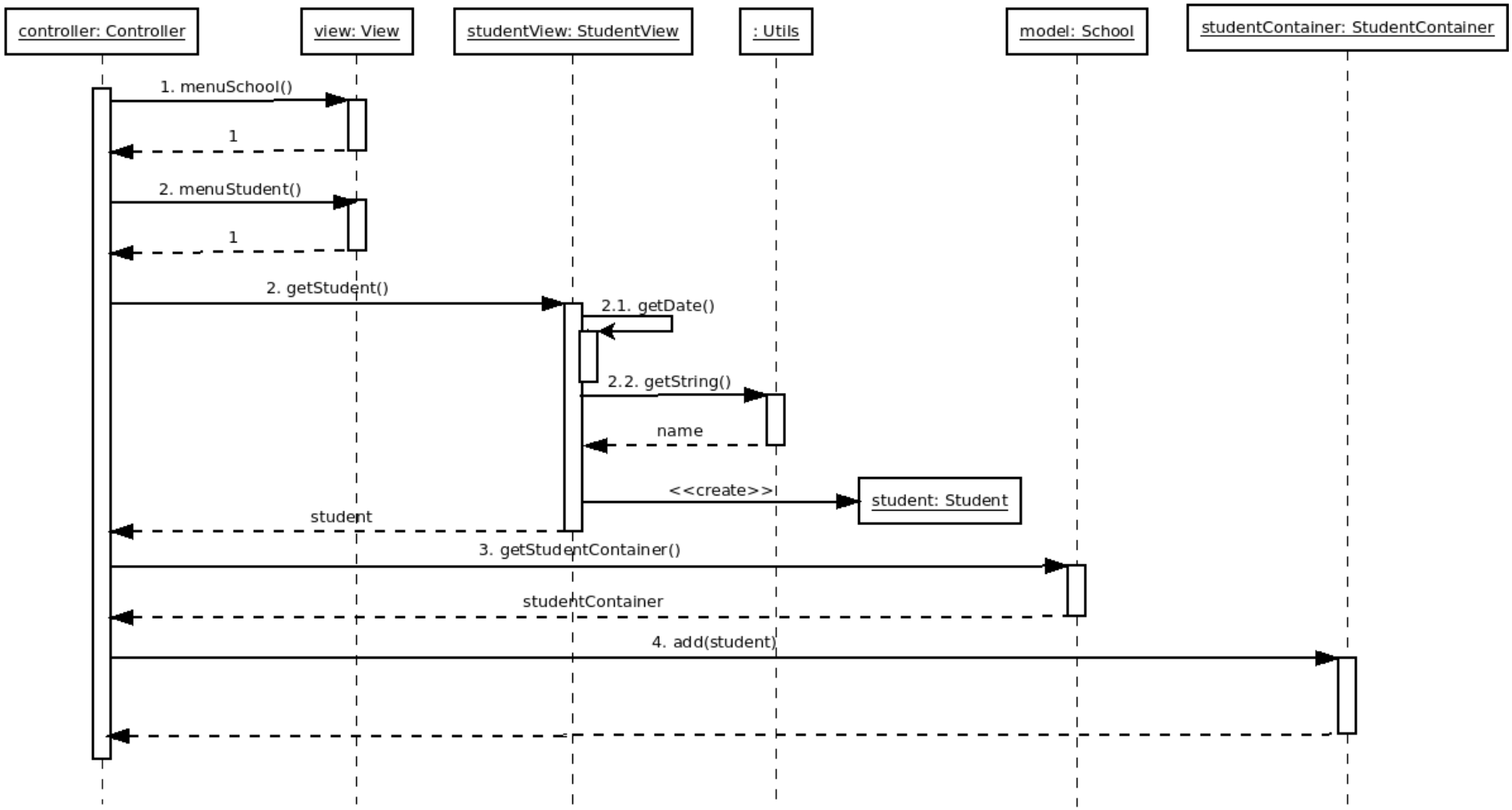
Design: `controllers` Class Diagram

- Usage of class diagram

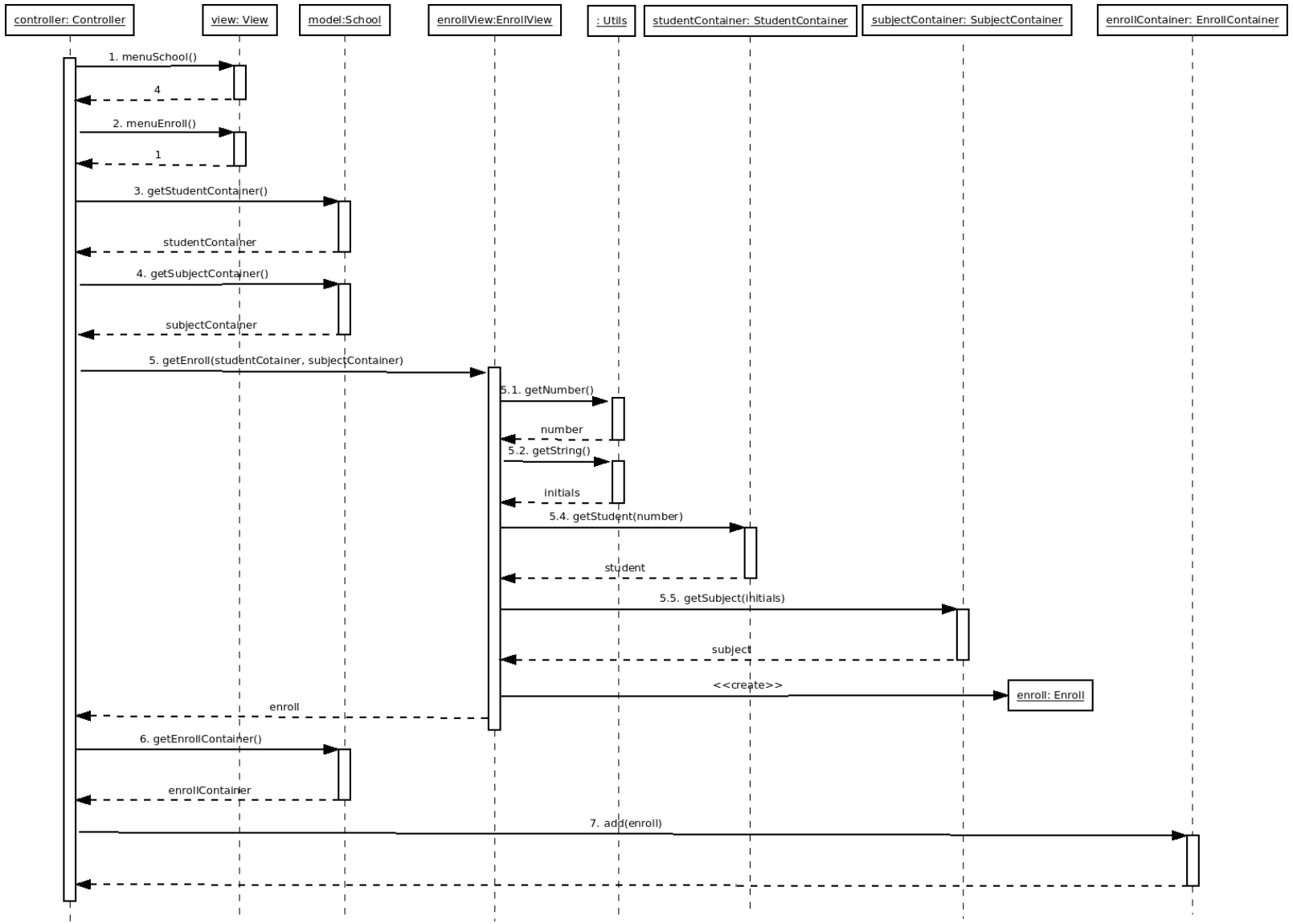


Design: UC 1 - Create Student

- Usage of sequence diagram



Design:UC 10 - Create Enroll



Design:User Interface

